Study Notes - Quadrotor Trajectory Tracking via Differential Flatness + Quaternions

Paper: K. Choutri, M. Lagha, L. Dala, M. Lipatov, "Quadrotors Trajectory Tracking using a Differential Flatness-Quaternion based Approach", IEEE 2017 - with worked maths, intuition, and the numbers from our repo's implementation (flatness/, 68-test suite, independently validated equation-by-equation).

1. The problem, and the paper's one big idea

A quadrotor is an underactuated system: it has 6 degrees of freedom (position + attitude) but only 4 control inputs (total thrust and 3 torques). You cannot command "move sideways" directly - the only way to accelerate horizontally is to tilt the vehicle so that the thrust vector (always along the body z-axis) points partly sideways. Attitude control alone (our base paper, Fresk & Nikolakopoulos 2013) therefore flies a drone that drifts wherever its tilt pushes it: excellent at pointing, blind to position.

To make the drone go somewhere you must invert this coupling: given where you want the vehicle to be (and how it should accelerate there), compute the thrust magnitude and the attitude that produce it. That inversion is exactly what this paper contributes, in three steps:

- Differential flatness - proves the quadrotor's full state and input are determined by 4 chosen outputs (x, y, z, yaw) and their derivatives. Planning becomes algebra, not simulation.
- Quaternion reference generation - converts the desired acceleration into the desired attitude quaternion q_d without ever passing through Euler angles, so there is no gimbal-lock singularity.
- Double-loop LQR - an outer position loop and an inner attitude loop, both optimal linear controllers computed from one Riccati equation.

> **How to read this paper**
>
> Sections II (modelling) and IV (LQR) are standard material - the paper's real content is Section III (flatness) plus Eqs. (19)-(21) (reference generation). If you understand the chain "acceleration -> thrust vector -> tilt -> quaternion -> LQR", you understand the paper. Everything below builds that chain piece by piece.

2. Quaternion toolkit (paper Eqs. 1-7)

2.1 What a quaternion is, and why we care

A quaternion is a 4-vector q = q0 + q1 i + q2 j + q3 k: one scalar part q0 and a 3-vector part q. A unit quaternion (||q|| = 1) represents a 3D rotation: compactly (4 numbers vs 9 for a rotation matrix), without singularities (unlike Euler angles, which break at pitch = + or -90 degrees), and with cheap, smooth composition (quaternion multiplication instead of matrix multiplication). The paper uses scalar-first order throughout: q = [q0, q1, q2, q3] with q0 the scalar. This convention matters later - the paper says q_pd's "4th element" is zero; in scalar-first order that is q3, the z-rotation (yaw) content.

$$q = q_0 + q_1 \cdot i + q_2 \cdot j + q_3 \cdot k = (q_0, \bar{q}) \quad \text{with } q_0 \in \mathbb{R},\ \bar{q} \in \mathbb{R}^3$$

Eq. (1): quaternion definition.

2.2 The operations (Eqs. 2-5)

Product (Eq. 2). Written with scalar/vector parts (p0, p̄) and (q0, q̄): the product is a new quaternion. Note it is not commutative - order encodes the order of rotations:

$$p \otimes q = (p0 \cdot q0 - \bar{p} \cdot \bar{q} , \; p0 \cdot \bar{q} + q0 \cdot \bar{p} + \bar{p} \times \bar{q})$$

Eq. (2): Hamilton product. The cross product term is what makes 3D rotations out of 4 numbers.

Conjugate, norm, inverse (Eqs. 3-5). $q^* = (q0, -\bar{q})$; $||q|| = \sqrt{(q \otimes q^*)}$; $q^{-1} = q^*/||q||$. For unit quaternions, $q^{-1} = q^*$ and conjugation reverses the rotation. Axis-angle (Eq. 5): a rotation by angle α about unit axis u is

$$q = [\cos(\alpha/2), \; u \cdot \sin(\alpha/2)]$$

Eq. (5): the axis-angle form - the single most useful identity for building reference quaternions by hand.

Rotation of a vector (Eq. 4). $v' = q \otimes [0, v] \otimes q^*$ rotates v by q. As a matrix, R(q) (Eq. 12's DCM) has orthonormal columns equal to the body axes expressed in world coordinates; then $v' = R(q) \cdot v$ and $R(q)R(p) = R(q \otimes p)$.

2.3 The kinematic equation (Eq. 6) - read this twice

$$\dot{q} = \tfrac{1}{2} \cdot q \otimes [0, \omega] \qquad (\omega \text{ in BODY frame})$$

$$\dot{q} = \tfrac{1}{2} \cdot [0, \omega_w] \otimes q \qquad (\omega_w = R(q) \cdot \omega \text{ in WORLD frame})$$

Eq. (6) and its world-frame twin. Same physical motion, two valid algebraic forms - you must match the form to the frame your ω lives in.

> **A trap our own validation caught (worth an exam question)**
>
> Our first plant implementation used the world-rate form ½[0,ω]⊗q while feeding it BODY rates (the same ω as the body-frame Euler equation $J \cdot \dot{\omega} = -\omega \times J\omega + \tau$). The two forms differ by exactly the term $[0, \bar{q}_v \times \omega]$: identical at hover, but on a 2-second, 2 rad/s spin test the wrong pairing accumulated 1.5 radians of attitude error while the correct pairing stayed at 3×10^{-8}. Rule: one ω symbol in one equation means one frame - check which.

2.4 Euler angles and gimbal lock (Eq. 7)

Eq. (7) converts between q and (φ, θ, ψ). The mapping is singular: at θ = ±90° the same physical attitude has infinitely many (φ, ψ) representations, and small attitude changes produce huge angle jumps. Euler angles are fine for plotting, dangerous inside a control law. The whole paper is organized to keep the loop in quaternion space and touch Euler angles only at the edges.

3. The vehicle model (paper Eqs. 8-10)

3.1 Newton-Euler with quaternions (Eq. 8)

State: position p, velocity ṗ, attitude quaternion q (body->world), body angular velocity ω. The four rows, each with its physical meaning:

$$\dot{p} = v$$

$$\dot{v} = R(q) \cdot [0,0,T/m] - [0,0,g] \qquad \text{thrust force minus gravity}$$

$$\dot{q} = \tfrac{1}{2} \cdot q \otimes [0, \omega] \qquad \text{(Eq. 6, body rates)}$$

$$J \cdot \dot{\omega} = -\omega \times (J \cdot \omega) + \tau_u + \tau_{ext} \qquad \text{(Euler's equation, body frame)}$$

Eq. (8): the complete 6-DOF plant. T is total rotor thrust, τ the body torques.

- Row 2 is the coupling - thrust always points along body-z, so the R(q) matrix decides where it points in the world. Tilt IS lateral acceleration.
- Row 4's $-\omega \times J\omega$ term is the gyroscopic effect: spinning bodies resist having their rotation axis turned. For our vehicle (Ixx = Iyy) it mostly couples pitch-rate into roll-rate at high spin.
- The paper assumes the centre of mass sits exactly at the body-frame origin - if not, gravity would create a torque and the attitude plant would change.

3.2 The rotor mixer (Eq. 9)

Four motors produce the four inputs. Each motor i contributes thrust $b \cdot \omega_i^2$ ($b = kf$) along body-z plus a reaction torque about body-z (sign alternating with propeller direction). Arranged in an X-configuration at arm distance d, the map from squared rotor speeds $w_i = \omega_i^2$ to (T, τ) is linear:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} b & b & b & b \\ bd & bd & -bd & -bd \\ -bd & bd & bd & -bd \\ -c & c & -c & c \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}$$

Eq. (9): the mixer matrix - X-config rows in order FR, FL, RL, RR; c = km (yaw reaction), d = arm/√2. Inverting it turns any (T, τ) command into rotor speeds: $w = M^{-1} \cdot [T, \tau]^T$ - every real flight controller contains exactly this matrix.

> **Saturation is where theory meets hardware**
>
> Rotor speeds cannot go negative or above a maximum. Naively clipping w_i to [0, w_max] after inverting Eq. (9) corrupts the TOTAL thrust one-sidedly (clipped rotors lose thrust, unclipped keep it) - our z-loop ran away to thousands of metres until we implemented priority desaturation: deliver thrust exactly, uniformly scale the torque until feasible. Same idea as real firmware. Measured, in our repo, with tests.

3.3 Hover linearization (Eq. 10)

The paper linearizes Eq. 8 at hover (q = identity, ω = 0). Expanding R(q) to first order in the vector part ($q \approx [1, \bar{q}/2 \text{ direction}]$), the horizontal acceleration becomes proportional to the small tilt:

$$\dot{v} \approx [\, 2 \cdot (T/m) \cdot q2 \;,\; -2 \cdot (T/m) \cdot q1 \;,\; T/m - g \,]^T$$

Eq. (10): at hover, tilt components q1, q2 steer; thrust T climbs. This linearization is what the inner-loop LQR of Eq. (17) is built on.

4. Differential flatness (Eqs. 11-15) - the heart

4.1 The definition, plainly

A system $\dot{x} = f(x, u)$ is differentially flat if there exists an output z(t) (the flat output) such that the state and input are functions of z and finitely many of its derivatives - with NO integration of the ODE:

$$x(t) = \zeta_1(\, z, \dot{z}, \ddot{z}, \ldots z^{(\alpha)} \,)$$

$$u(t) = \zeta_2(\, z, \dot{z}, \ddot{z}, \ldots z^{(\beta)} \,)$$

Eqs. (11)-(12): flatness definition. If you can write these two maps, planning in x/u space is equivalent to planning in z space - which is much smaller.

Why that is a gift: to find a trajectory you no longer guess inputs and simulate. You pick z(t) directly (any smooth curve you like), differentiate it on paper, and read off the exact state and

input that fly it. Constraints on the vehicle (max thrust, max tilt) become constraints on z's derivatives - the optimization happens in the small output space.

4.2 The quadrotor's flat outputs: σ = (x, y, z, ψ) (Eq. 13)

Four inputs, four flat outputs - the position and the yaw. Intuition for WHY these determine everything, in a chain (this chain is Eq. 15, and it is the sentence to remember from this paper):

σ = (x, y, z, ψ)	Eq. (13)
p = (x,y,z), v = ṗ, a = p̈	Eq. (14) (trivial rows)
f = m · (a + g·ẑ)	thrust VECTOR (Eq. 15)
T_d = ‖f‖ , d = f/T_d	magnitude + direction
R(q_d)·ẑ = d and yaw(q_d) = ψ	fixes q_d uniquely
ω_d = q_d⁻¹ ⊗ 2·q̇_d ...	differentiate for rates
τ = J·ω̇_d + ω_d × J·ω_d	input parameterization

Eq. (15) in words: outputs -> accelerations -> thrust vector -> attitude -> rates -> torques. Every state and input, algebraically.

- Position rows are trivial (Eq. 14) - that is the point: flatness makes the hard part explicit rather than hidden in dynamics.
- The thrust-vector row f = m(a + g·ẑ) comes straight from Eq. 8's row 2: if you WANT acceleration a, you NEED net force m·a, and the only forces are thrust and gravity, so the thrust vector is fully determined.
- Attitude: the direction d = f/‖f‖ tells you where body-z must point (2 constraints of the 3-DOF attitude); the chosen yaw ψ fixes the remaining degree of freedom. Hence q_d is uniquely determined by (a, ψ).
- The only singularities of the whole chain: f = 0 (free fall - attitude is then irrelevant to the motion) and, in Euler-based versions, gimbal lock. The quaternion formulation removes the second one entirely.

> **Flatness in one sentence**
>
> A quadrotor cannot be commanded sideways, but it CAN be commanded to accelerate sideways - if you accept tilting. Flatness is the bookkeeping that makes that acceptance exact: choose (x,y,z,ψ)(t), and the required tilt, thrust, rates and torques all fall out of algebra on derivatives.

5. Generating the reference attitude (Eqs. 19-21)

The outer loop outputs the desired acceleration a_d. The flatness chain turns it into the attitude reference q_d that the inner loop will track. The paper does this in two clean pieces and composes them:

```
q_pd = tilt quaternion taking ẑ onto d = f/‖f‖

= [cos(α/2), sin(α/2)·(ẑ×d)/‖ẑ×d‖],   α = angle(ẑ, d)

-> 4th element (z/yaw content) is EXACTLY zero      (Eq. 19)

q_zd = [cos(ψ_d/2), 0, 0, sin(ψ_d/2)]                (Eq. 20)

q_d  = q_pd ⊗ q_zd                                   (Eq. 21)
```

The tilt quaternion's rotation axis ẑ×d is horizontal by construction, so its z-component (the paper's "4th element") is identically zero - that is all Eq. 19's remark means. q_zd is a pure yaw. The composition is a tilt-then-yaw cascade.

Worked micro-example (numbers from our implementation, m = 0.2 kg). Want a_x = +2 m/s², level, yaw 0: f = 0.2·(2, 0, 9.81) = (0.4, 0, 1.962) N; T_d = ‖f‖ = 2.002 N; d = (0.1997, 0, 0.9799); α = 11.5° about axis (0,1,0) (pitch back); q_pd = [0.9950, 0, 0.0997, 0] - note the exact zero in the 4th slot; q_zd = identity; so q_d = q_pd: a pure 11.5° pitch. Feed (T_d, q_d) forward and the drone accelerates at exactly 2 m/s².

Two ways to build the same idea - and why it mattered to us

We first built q_d by Gram-Schmidt (complete d to a right-handed frame keeping the heading, then DCM->quaternion). An independent review proved the two objects differ by a twist about the thrust axis that grows QUADRATICALLY with tilt (0.03° at our circle's 2.8° tilt, 2° at 45° tilt) - and our code comment claiming exact equivalence was wrong. We now implement Eqs. (19)-(21) literally: measured z-component of q_pd = -5.6×10^{-17}. Lesson: "equivalent constructions" deserve a numerical test, not a comment.

6. The double-loop LQR (Eqs. 16-23)

6.1 Why two loops, and why LQR

The plant is naturally cascaded: position responds slowly THROUGH attitude, attitude responds fast to torque. A double-loop structure matches it: an outer position loop computes the acceleration command (which the flatness map turns into T_d and q_d), and an inner attitude loop tracks q_d with torque. The inner loop must be roughly an order of magnitude faster - the classic time-scale separation rule. LQR (Linear Quadratic Regulator) is chosen because for a linearized model it is provably optimal for a quadratic cost and gives guaranteed stable closed-loop poles - and the gains come from one matrix equation instead of hand-tuning three per axis.

6.2 The two linear models (Eqs. 17-18)

```
INNER (attitude):  state x = [q_vec(3), ω(3)]

      A = [ 0₃ₓ₃   ½I₃ ]      B = [ 0₃ₓ₃ ]

[ 0₃ₓ₃   0₃ₓ₃ ]          [  J⁻¹  ]            (Eq. 17)

OUTER (position, per axis):  state x = [p, ṗ]

A = [ 0  1 ]    B = [ 0 ]                       (Eq. 18)

    [ 0  0 ]        [ 1 ]     (input = acceleration)
```

Inner A comes straight from Eq. 10's linearization ($\dot{q}_{vec} \approx \omega/2$, $\dot{\omega} = J^{-1}\tau$); outer is the double integrator. Both are the paper's Eq. 16 form $\dot{x} = Ax + Bu$.

6.3 The optimal gain (Eqs. 22-23)

$$J = \tfrac{1}{2}\int_0^{\infty}\left(x^T\cdot Q\cdot x + u^T\cdot R\cdot u\right)dt \qquad \text{(Eq. 22)}$$

$$P\cdot A + A^T\cdot P - P\cdot B\cdot R^{-1}\cdot B^T\cdot P + Q = 0 \qquad \text{(algebraic Riccati)}$$

$$K = R^{-1}\cdot B^T\cdot P\ , \qquad u = -K\cdot x \qquad \text{(Eq. 23)}$$

Q penalizes state error (how tightly to track), R penalizes effort (how cheaply). Solve the Riccati equation once offline; K is the constant gain matrix the loops apply at runtime. scipy.linalg.solve_continuous_are does exactly this in our lqr.py.

- Q large vs R small -> aggressive tracking, high actuator use; the reverse -> lazy but cheap. Our documented tuning: outer (49, 42; R=1) per axis, inner (100, 20; R=200·I). The paper gives NO numbers ("tuned adaptively") - any reproduction must name its own, and ours are in the README.
- For the outer double integrator, LQR is still a PD law with optimal gains: a_cmd = a_ref − Kp·(p−p_ref) − Kd·(v−v_ref) - the flatness feedforward a_ref does the work of an integral term without one.
- The paper's Fig. 2-3 wiring: altitude part of the outer loop outputs T_d directly; lateral part outputs q_pd; inner loop outputs τ; Eq. 9 mixes to rotor speeds. One 200 Hz controller rate for the whole cascade (Section V).

7. The study case, our reproduction, and what we learned building it

7.1 The paper's experiment (Section V)

Circle at 1 m altitude starting from p0 = (−1, 0, 0), controller at 200 Hz, actuator and battery limits "taken into consideration" (no numbers given). Reported: X/Y follow with about 1 s lag due to altitude settling (Figs. 4-5), clean Z response (Fig. 6), full 3D tracking (Fig. 7), PWM signals within ESC range (Fig. 8), quaternions stable around identity (Fig. 9). Note the paper's own inconsistency: "1 m of diameter" cannot pass through (−1,0,0) - we fly radius 1 m, resolving in favour of the explicit start point.

7.2 Our measured results (repo: flatness/, deterministic, seed 0)

Plant	RMS x	RMS y	RMS z	RMS radial	Peak rotor
Paper Eq. (8) ideal model	1.6 cm	3.6 cm	0.6 cm	2.7 cm	0.57 N
MuJoCo contact physics, 20 kHz	1.4 cm	3.7 cm	0.7 cm	2.8 cm	0.99 N
MuJoCo + paper noise ±0.1	2.3 cm	4.6 cm	0.9 cm	3.7 cm	1.69 N

Approximately 97% radial accuracy on the 1 m circle, no rotor saturation (1.95 N limit), and near-identical numbers on two independent plants - which is the flatness map doing its job: the reference is exact regardless of the integrator behind it.

7.3 Three engineering findings you can quote in a viva

- Finding - naive rotor clipping diverges - one-sided clipping corrupts total thrust; thrust-exact torque desaturation (uniform scaling) fixes it. Proven by watching the ideal model run away to thousands of metres before the fix.
- Finding - the actuator lag is load-bearing - with a realistic 20 ms ESC first-order lag the 200 Hz ZOH loop is stable; remove it and the lateral mode diverges (the ideal model diverges identically - proven). Real motors do not change thrust instantly; the paper's qualitative "ESC

limits" sentence is quantitatively essential.

- Finding - LQR weights are yours to choose - the paper's "adaptive tuning" gives no transferable numbers; we document a tuning procedure and its result (position bandwidth ~7 rad/s, attitude ~23 rad/s, 3x separation).

8. One-page formula sheet

Object	Formula
Quaternion product	$p \otimes q = (p_0 q_0 - \vec{p}\cdot\vec{q},\ p_0\vec{q} + q_0\vec{p} + \vec{p}\times\vec{q})$
Axis-angle	$q = [\cos(\alpha/2),\ u\cdot\sin(\alpha/2)]$
Kinematics (body rates)	$\dot{q} = \tfrac{1}{2}\, q \otimes [0,\ \omega]$
Plant	$\dot{v} = R(q)[0,0,T/m] - g\hat{z}$; $J\dot{\omega} = -\omega\times J\omega + \tau$
Mixer	$[T,\ \tau]^T = M\cdot[\omega_i^2]$, $w = M^{-1}[T,\ \tau]^T$, clip/desaturate
Hover linearization	$\dot{v} \approx (2T/m)[q_2, -q_1, \ldots]$, $\dot{q}_{vec} \approx \omega/2$
Flat outputs	$\sigma = (x, y, z, \psi)$
Thrust vector	$f = m(a + g\hat{z})$, $T_d = \|f\|$, $d = f/T_d$
Tilt quaternion	$q_{pd} = [\cos(\alpha/2),\ \sin(\alpha/2)\cdot(\hat{z}\times d)/\|\hat{z}\times d\|]$, $\alpha = \angle(\hat{z}, d)$
Yaw quaternion	$q_{zd} = [\cos(\psi/2),\ 0,\ 0,\ \sin(\psi/2)]$
Attitude reference	$q_d = q_{pd} \otimes q_{zd}$
Inner LQR model	$[q_{vec},\ \omega]$ with $A=[[0, I/2],[0,0]]$, $B=[0; J^{-1}]$
Outer LQR model	$[p,\ \dot{p}]$ double integrator, input = accel
Riccati / gain	$PA+A^TP-PBR^{-1}B^TP+Q=0$, $K=R^{-1}B^TP$, $u=-Kx$
Loop wiring	pos LQR -> a_cmd -> flatness -> (T_d, q_d) -> att LQR -> τ -> mixer

9. Self-test (cover the right column)

Question	Answer
Why exactly 4 flat outputs for a quadrotor?	4 independent inputs (T, τx, τy, τz); flat outputs must match the input rank - (x, y, z, ψ) is the canonical choice.
Where does $f = m(a + g\hat{z})$ come from?	Newton's law on Eq. 8 row 2: desired net force = m·a; only thrust + gravity act, so the thrust vector is fully determined.
Why is q_pd's 4th element zero?	Its rotation axis $\hat{z}\times d$ is horizontal, so the z (yaw) component of the quaternion vanishes identically - a tilt with no yaw content.
What breaks at thrust = 0?	The direction $d = f/\|f\|$ is undefined (free fall) - attitude stops influencing the translational motion. The only true singularity of the map.
Body vs world form of $\dot{q}$ - which is which?	$q \otimes [0,\omega]$ pairs with BODY rates; $[0,\omega_w] \otimes q$ with WORLD rates. Mixing them drifts as O(tilt × rate) - our validation measured 1.5 rad in 2 s.
Why two LQR loops instead of one big one?	Time-scale separation: attitude must be ~10x faster than position; separate linearizations (Eq. 17 vs 18) and gains keep each loop optimal where it operates.
What do Q and R penalize, practically?	Q: how hard to fight state error per axis; R: the price of actuator effort. Raising inner R softens torque commands (keeps the mixer unsaturated).

Question	Answer
Why does the outer loop need no integral term?	The flatness feedforward a_ref supplies the exact required acceleration; the LQR only removes disturbance/error - steady-state error is zero by construction on exact references.
What is the paper's honest gap?	No numeric gains, no trajectory timing, no disturbance tests - and a diameter/start-point inconsistency in the study case. Reproductions must document their own choices (ours: README assumptions table).
One sentence: what does flatness buy a planner?	Plan in (x,y,z,ψ) with any smooth curves; the vehicle's exact state and inputs are recovered by differentiating - no simulation in the loop.

10. Glossary

- Flat output - the variable set whose derivatives parameterize all states and inputs; here (x, y, z, ψ).
- Underactuated - fewer independent inputs than degrees of freedom (4 vs 6).
- Zero-order hold (ZOH) - the command stays constant between control ticks; at 200 Hz that 5 ms stair-step is part of the loop dynamics.
- Riccati equation - the matrix quadratic whose solution P yields the LQR gain; the steady-state (algebraic) form fits infinite-horizon cost.
- Time-scale separation - inner loop much faster than outer, so each sees the other as quasi-static.
- Desaturation - recovering from actuator limits by scaling commands uniformly while preserving what matters (here: exact thrust).
- ESC lag - first-order motor/thrust response delay; 20 ms in our documented model.
- Gimbal lock - Euler-angle singularity at pitch ±90°; the reason the reference path stays in quaternions.

Sources: the paper (IEEE 2017); our implementation flatness/ (validated 17/17 findings by independent review); repo README section "Path planning" for every assumption and result table.